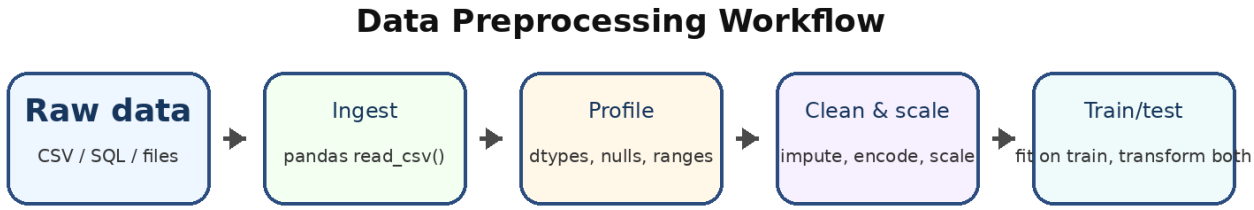
Data Preprocessing

Clean, structured notes created from the handwritten pages



Quick idea

Before training an ML model, convert raw data into a reliable feature matrix: correct dtypes, handle missing values, scale numeric columns, and split train/test correctly.

1. What is data preprocessing?

Data preprocessing is the set of steps used to prepare raw data for model training. In the handwritten notes, the flow starts with machine learning and model training, then moves into pandas ingestion, dataframes, features, targets, dtype checks, missing values, scaling, and train/test splitting.

In pandas, data is typically ingested into a DataFrame, often named df. The columns are split into features X and a target y before training a model.

2. Practical preprocessing checklist

Step	What to do
Ingest data	Load CSV/SQL/API data into a pandas DataFrame, e.g., pd.read_csv().
Profile the data	Check shape, columns, dtypes, missing values, unique values, ranges, and outliers.
Fix dtypes	Make sure each column has the correct type: numeric, category, text, datetime, boolean.
Handle null values	Remove or impute missing values using mean, median, mode, or domain-specific values.
Scale numeric features	Put numeric columns on comparable scales using StandardScaler, MinMaxScaler, RobustScaler, or Normalizer.
Split data	Create train/test sets. Fit preprocessing only on training data, then transform train and test.

3. Data profiling with pandas

Profiling means understanding what is inside the dataset before changing it. The notes include a SQL-style filter and pandas checks such as dtypes and isna().

```
df.head()              # preview rows
df.info()              # dtypes and non-null counts
df.describe()          # numeric summary
df.isna().sum()        # missing values per column
df[df['homeowner'] == 'Y'] # example filter
```

Useful profiling commands before modeling.

Why dtypes matter

A model cannot safely learn from a messy mixture of strings, dates, numbers, and categories unless those columns are represented correctly.

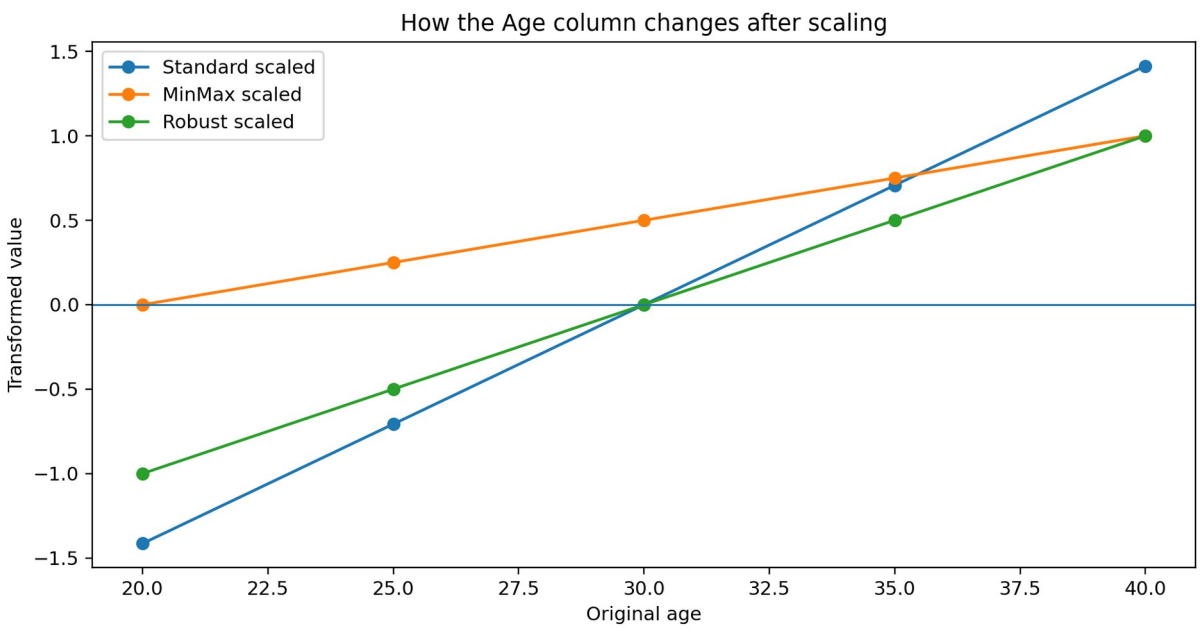
4. Missing values

The notes mention removing null values or replacing them using mean, median, mode, or standard deviation-based logic. In practice, choose the method based on the column type and business meaning.

Column type	Common strategy	Example
Numeric, roughly symmetric	Mean imputation	salary.fillna(salary.mean())
Numeric with outliers	Median imputation	age.fillna(age.median())
Categorical	Mode imputation	city.fillna(city.mode()[0])
Too many missing values	Drop column or collect data	df.drop(columns=[...])

5. Feature scaling overview

Scaling makes numeric features comparable. This is important when columns have very different units, such as Age and Salary. The handwritten example uses Age = 20, 25, 30, 35, 40 and Salary = 30, 50, 60, 70, 90.



5.1 StandardScaler

StandardScaler centers data around the mean and measures values in standard deviations. It is useful for many linear models, SVMs, KNN, PCA, and gradient-based methods.

x' = (x - mean) / standard_deviation

For Age: mean = 30 and standard deviation is about 7.071. So age 20 becomes about -1.414.

5.2 MinMaxScaler

MinMaxScaler squeezes values into a fixed range, usually [0, 1]. It keeps the original ordering but changes the scale.

$$x' = (x - min) / (max - min)$$

For Age: min = 20 and max = 40. Age 35 becomes (35 - 20) / 20 = 0.75.

5.3 RobustScaler

RobustScaler uses the median and interquartile range, so it is more resistant to outliers than StandardScaler.

$$x' = (x - median) / IQR$$

IQR = Q3 - Q1. For Age: median = 30, Q1 = 25, Q3 = 35, so IQR = 10.

5.4 Normalizer

Normalizer works per row, not per column. It scales each sample so the row vector has length 1. This is common when direction matters more than magnitude, such as text vectors or cosine-similarity workflows.

Normalizer: each row becomes a unit vector

Row	Before	After L2 normalization
Sample 1	[20, 30]	[0.555, 0.832]
Sample 2	[25, 50]	[0.447, 0.894]
Sample 3	[30, 60]	[0.447, 0.894]

Formula: x_new = x / ||x||2, where ||x||2 = sqrt(sum(x_i^2))

6. Worked example: Age and Salary

Age	Salary	Age z-score	Age min-max	Age robust	Salary robust
20	30	-1.414	0.00	-1.0	-1.5
25	50	-0.707	0.25	-0.5	-0.5
30	60	0.000	0.50	0.0	0.0
35	70	0.707	0.75	0.5	0.5
40	90	1.414	1.00	1.0	1.5

Notice how the different scalers create different transformed values while preserving the order of the original Age column.

7. Train/test split and leakage

The notes end with fitting the scaler on training data and transforming both training and test data. This is the correct pattern because the test set must stay unseen during fitting.

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)  # fit only on train
X_test_scaled = scaler.transform(X_test)        # transform test using train stats
```

Never fit preprocessing on the full dataset before the split. That leaks test information into training.

Memory tip

fit_transform on training data; transform on test data. The scaler learns from train only.

8. Which scaler should I choose?

Scaler	Best when	Watch out for
StandardScaler	Features are roughly bell-shaped; model is distance/gradient sensitive.	Sensitive to strong outliers.
MinMaxScaler	Need values in a fixed range like [0, 1].	Very sensitive to min/max outliers.
RobustScaler	Dataset has outliers or skewed numeric columns.	May not center/scale the same way as standardization.
Normalizer	Each row is a vector and direction matters.	It is row-wise, not column-wise.

Appendix: original handwritten references

The pages below were included as visual references from the uploaded handwritten notes.

Handwritten page 1

16 May 2026 09:06

Data preprocessing -

ML → Model training

→ pandas ingest dataFrame → df

features target

→ dtype match according to the data in the columns

→ no null values (Remove) → mean, median, std

→ Scaling (4 columns in same scaling)

→ train, test

profiling of data.

Select * from ~~horse power~~ mpg where horsepower = '?'

→ df [df ['horsepower'] == '?']

inject into df properly → read_csv

dtypes → mismatch - Solve

isna → ✓ (SP) mean, median, std

New Section 1 Page 1

Handwritten page 2

isna → ✓ (SP) mean, median, std

| 7 |

1) standard scalar
2) minmax scalar
3) Robust scalar
4) Normaliser

Age	salary (K)
20	30
25	50
30	60
35	70
40	90
(X)	(x)

X

Age: 20, 25, 30, 35, 40

$x' = (x - \mu)/\sigma$

$\mu = (20 + 25 + 30 + 35 + 40)/5 = 30$ ✓

$\sigma_{age} = \sqrt{(-10)^2 + (-5)^2 + 0^2 + (5)^2 + 10^2}$

$= \sqrt{100 + 25 + 0 + 25 + 100}$

Handwritten page 3

$= \sqrt{250/5} = \sqrt{50} = 7.0711$

Age	$x-\mu$	$\div \sigma$	$x' = (x-\mu)/\sigma$
20	-10	7.0711	$= -10/7.0711 = -1.42$
25	-5	7.0711	$= -0.7131$
30	0	7.0711	0
35	5	7.0711	0.7131
40	10	7.0711	1.42

2) <u>MinMaxScalar</u>: "squeeze everything into [0,1]"

Formula: $x' = (x-\min)/(\max-\min)$

in bounded-range [-1, 1]

image pixel (R, G, B) → [0, 1]

0 0 0

255 255 255

Age	Salary
20	30
25	50
30	60
35	70
40	90

Age – column → x = 20, 25, 30, 35, 40

min = 20 max = 40

Age (x)	x − min	max − min	(x − min) / (max − min)	x'
20	0	20	0/20	0
25	5	20	5/20	0.25
30	10	20	10/20	0.5
35	15	20	15/20	0.75
40	20	20	20/20	1

(8) Robust Scaler: "center on median spread IQR"

Formula = $x' = (x - \text{median}) / IQR$

where $IQR = Q_3 - Q_1$

Age sorted = 20, 25, 30, 35, 40 (Q_1 = 25, Q_3 = 35; median = 30)

median = 30, $Q_1 = 25$, $Q_3 = 35$, $IQR = 10$

Salary sorted = 30, 50, 60, 70, 90 (Q_1 = 50, Q_3 = 70; median = 60)

median = 60, $Q_1 = 50$, $Q_3 = 70$ $IQR = 20$

Age (x)	x − 30/10	x'	salary (x)	x − 60/20	x'
20	−10/10	−1	30	−30/20	−1.5
25	−5/10	−0.5	50	−10/20	−0.5
30	0/10	0	60	0/20	0
35	[illegible]		[illegible]		

Handwritten page 5

30	0/10	0	60	0/20	0
35	5/10	0.5	70	10/20	0.5
40	10/10	1	90	20/20	1.5

$2^0 = 1 + 2^{1-1} = 2^1 \times 2^0$

$= 2/2 = 1$ TOR

(4) Normalizer: make each row a unit vector

per-row, per-sample

$L_2 = x' = x / \|x\|_2$

$x_i = x_i / \|x\|_2$ where $\|x\|_2 = \sqrt{(\Sigma x_i^2)}$

Age	Salary
20/36.055	30/36.055
25/55.90	50/55.90
30/67.05	60/67.05
35/78.26	70/78.26
40/98.05	90/98.05

36.055

$\sqrt{20^2+30^2} = \sqrt{1300} = 36.055$

$\sqrt{25^2+50^2} = \sqrt{625+2500} = \sqrt{3125}$

$\sqrt{30^2+60^2} = \sqrt{900+3600} = \sqrt{4500}$

$\sqrt{35^2+70^2}$

$1225 + 4900$

6100

1600

raw data → preprocessor → Scaler → Λ

Handwritten page 6

randate → preprocessor → Scaler → fetur →

minmax → cheapest

normalizer →

standardscaler → modelete

Robust scaler → explorer

```
scaler = standscaler()
scaler.fit(X)
X1new = scaler.fit(n)
x_train = scaler.fit_transform(x_train)
x_test = scaler.transform(x_test)
```

X

x_train 80%